Known-Contact Recognition Assistant

1. Overview and Motivation

This project outlines the technical implementation of real-time object detection along with face recognition to assist a person in recognizing people and hazards around them. The target use case is assistive, i.e. helping someone with face blindness (prosopagnosia), low vision, or memory difficulty recognize people they already know in everyday settings such as a hallway, campus walkway, or social event.

The system runs on a standard video feed (webcam or recorded clip) and overlays real-time information. The classification includes a bounding box and distance estimate for every detected person, cat, or dog, and, for people the user already knows, a small information panel with their name and a short contextual note.

2. The Evolution of the Architecture & Implementation Strategy

The system was developed iteratively, progressing, from basic computer vision to a simulated spatial computing environment:

Phase 1: Basic Object Detection: I started the project with a standard object detection pipeline. I wanted to identify hazards (people, cats, dogs) in a video feed, building upon my undergraduate script I had developed previously. YOLOv8-nano (Ultralytics), detects three classes relevant to the scenario: person (class 0), cat (class 15), and dog (class 16). No training or fine-tuning has been done here. Because the pretrained weights already cover these categories.

Phase 2: Proximity-Based Alert System: After implementing standard object detection pipeline, I thought of moving forward to spatial context as detection alone didn't do anything useful. I implemented a ground plane assumption model. For measuring the distance, firstly I took all the values of the bounding box against the camera threshold. But it creates ambiguities between adults, children or pets of different heights. To solve this issue, I've tracked only the y_2 bounding box coordinate (where the subject's feet touch the ground). This avoids needing a per-class assumed real-world height.

Phase 3: Face Recognition and Context-Aware Social Memory: To make the system more interactive, I tried to integrate a cognitive-assistance tool. I wanted the system to say who it is. So, I shifted the purpose to recognizing pre-enrolled people and showing a short note about them (context panel), like a memory aid for someone with face blindness or low vision. While detecting faces, I have manually added their reference image and summary into the database (`KNOWN_CONTACTS`) before running the system. A person who is not in that list is never flagged, labeled "Unknown," or visually distinguished from anyone else. They just only get the same plain box and distance as every other detected person. This was a deliberate design choice by me so the system is scoped to be a personal memory aid among known contacts.

Phase 4: Performance Optimization: While implementing the system, optimization also came to my mind as running continuous facial recognition can be too heavy for wearable hardware. So, I used a lightweight object detection (YOLOv8n) runs continuously on edge devices, while heavy facial recognition only triggers when a person breaches a 2.5-meter social distance threshold.

Phase 5: Adaptive UI Panel Placement: After running the above system, I found that the static UI panels overlap and block the user's view. Rather than statically placing the panel, the system evaluates six priority candidate coordinates (Right, Left, Bottom, Top, and diagonals) around the target person. It dynamically assigns the context panel to the first vacant location that does not intersect with any occupied space. If none fit, it falls back to a clamped position below the box. After the implementation I noticed the context panel sometimes goes further away from the recognized person. So, I added a tether line connecting the panel back to the recognized person to prevent ambiguity.

Phase 6: Separate Handling for Animals: For adding multiclass classification, I have labelled the cats and dogs with a distinct color and just a simple box-and-distance label. They are intentionally excluded from the face-recognition path.

3. Final Features

- Real-time detection of people, cats, and dogs (YOLOv8-nano, pretrained)
- Approximate distance per detection from the bottom edge of its box.
- Face recognition only for enrolled, known contacts, and at close range.
- No overlap with information boxes, labels, or other panels.

4. Transparency: Human-AI Collaboration

For research transparency, I used an AI assistant (Gemini) throughout this project. This section lists what it was used for, which decisions it suggested, and which were mine.

4.1 What I Handled (Decisions and work that were mine)

- **The idea and its architectural strategy:** The idea generation came from an undergraduate project I had done earlier, and my interest in integrating particular features (Context-Aware Social Memory/Adaptive UI placement) was inspired by Dr. Davari's iAR work. The decision to separate lightweight YOLO tracking from event-driven Face recognition to simulate hardware constraints along with the object classes (Person, cat, and dog) was given by me.
- **Distance Method:** Choosing the ground plane assumption rather than relying on standard bounding-box area calculations. The close-range trigger for face recognition and the rule that animals skip it.
- **Facial Recognition:** Deciding not to run facial recognition on strangers, cats and dogs, but strictly limiting it to a predefined contacts(KNOWN_CONTACTS). I optimized the placement logic to be adaptive to the frame and changed the fonts and colors to make labels readable at distance. I asked for a simple, real-time design rather than more advanced extensions.
- **UI Occlusion Logic:** Designing the priority-based candidate placement logic (Right, Left, Bottom, Top and diagonals) and the tethering line mechanic for the adaptive panel placement. I chose the sample test footage, ran every version on it, and identified the problems that drove the next iteration.

4.2 How I Utilized AI (Tooling & Integration)

- **Code structure:** I gave the assistant my own earlier object-detection script as a reference and asked it to produce a well-structured version for the new objective, rather than generating a project from nothing. I searched for face recognition libraries myself and also asked the assistant to compare options for a real-time prototype.
- **Debugging:** Used AI to quickly resolve OpenCV tensor format mismatches and array indexing errors during the FaceNet cropping phase. Errors such as the missing face_recognition module) and the panel-overlap and readability problems were worked through with prompts.
- **Documentation & Proofreading:** Used AI to refine the grammar and flow of my project summaries and proofread it myself.

4.3 Sample prompts

Below I've attached some real prompts from the project, lightly edited for typos. They show the pattern I followed: I described a problem or constraint, reviewed what comes back, tested it on video, and then decided what to change.

- Here is my previous object detection script. I want to build a prototype that does basic objection along with drawing bounding boxes for classes 0 (person), 15 (cat), and 16 (dog). Please refactor this to use YOLOv8n.
- I want to calculate the distance from the camera to the people who are walking towards the camera. Provide the updated Python script using YOLOv8-nano that tracks the bottom edge of the bounding box to trigger alerts, ignoring the height variation of the pedestrians.
- I want to elevate the system by integrating facial recognition of the known person in the contact list. I don't have a large database of known people, so I will enroll two or three sample faces from reference images, just to produce sample output for evaluating the project. And the system must exclude cats and dogs from face recognition path. When a known person is recognized, show a short information panel with their name and a short note. People who are not in the list should look like any other detected person, with no "unknown" label. Keep it simple but efficient in real time. No complex ideas and no higher-level redesign, just small updates to my existing code.
- I want the system to be lightweight, so I plan to use the dlib-based face recognition library. For example- by running it only on person crops at close range rather than on every frame. Give a final verdict of this code.

5. Sample input & sources

The three test clips are free-license stock footage from Pexels:

- https://www.pexels.com/video/two-women-dog-lovers-walking-their-pets-on-an-unpaved-road-4267741/

- https://www.pexels.com/video/adorable-ginger-cat-exploring-outdoors-29860176/
- https://www.pexels.com/video/officemates-walking-on-the-street-5311355/

Reference photos
For this demo, the two enrolled "contacts" (`XYZ`, `ABC`) were made from screenshots of faces in the test footage, not from real acquaintances who opted in. I did this to check that the whole pipeline (detect, match, draw panel) works end to end.
Sample screenshot from test footage:


6. Known Limitations

- The distance formula measures a level camera and flat ground. A tilted camera or slopped clip will reduce the result accuracy.
- Face recognition accuracy drops at long distance and in poor lighting.
- Fast motion or a moving camera may cause mismatches while face recognition.
- No formal accuracy evaluation has been done in this project.